

TRACE: Transfer-Aware Carbon-Efficient Scheduling for Partitioned DNN Inference at the Edge

Ebaad Raheem, Danish Naseer, Saad Ahmed Khan
Department of Computer Science
University of the Punjab, Gujranwala Campus, Pakistan
{bcs23033, bcs23006}@pucc.edu.pk

Abstract—Carbon-aware edge schedulers typically model only computational carbon, ignoring the energy cost of inter-node data transfer. This creates a *Transfer Trap*: schedulers route data over energy-intensive wireless links to reach nominally low-carbon nodes, often worsening both carbon and latency. We present TRACE, a scheduler that jointly accounts for compute and transfer carbon using per-link energy coefficients, Min-Max normalization, and a local-execution rule. In a SimPy simulation on a six-node heterogeneous testbed profiled with real PyTorch measurements, TRACE achieves up to 35% additional carbon reduction over Green-only scheduling on transfer-heavy models, while improving latency by up to 24% on network-light models.

Index Terms—edge computing, carbon-aware scheduling, DNN partitioning, transfer-aware scheduling, energy efficiency.

I. INTRODUCTION

Edge computing brings DNN inference closer to data sources [1]. This reduces latency but increases energy use across distributed nodes [2]. Existing schedulers like CarbonCP [3] and CarbonEdge [4] focus on computational carbon alone. They do not account for the energy needed to move data between nodes.

Network links are not equal. 4G LTE uses about 0.14 kWh/GB. Fiber uses about 0.04 kWh/GB [7]. A scheduler that ignores this difference might send data over a 4G link to reach a low-carbon node. The transfer carbon from that decision can exceed the compute savings. We call this the *Transfer Trap*.

We make four contributions:

- 1) A carbon model that combines compute energy (scaled by node CPU) and per-link transfer energy (4G LTE, 5G, WiFi, Fiber).
- 2) An $O(N)$ scheduling algorithm with five weighted scores and Min-Max normalization. A local execution rule assigns zero transfer cost when a partition stays on the same node.
- 3) A SimPy-based simulation of six heterogeneous edge nodes with real PyTorch profiling data.
- 4) Results showing that TRACE reduces latency for network-light models and keeps per-inference carbon low across MobileNetV2, ResNet-50, and VGG16.

II. RELATED WORK

Carbon-aware scheduling. The growing energy consumption of deep learning has motivated sustainable ML practices [11]. Schwartz et al. [13] introduced the Green AI paradigm, advocating for model efficiency as a primary evaluation metric. CarbonCP [3] uses conformal prediction to partition DNNs for lower compute carbon, reporting up to 58.8% reduction in compute emissions. CarbonEdge [4] introduces a framework for sustainable edge inference, achieving up to 22.9% reduction versus monolithic execution. An earlier CarbonEdge variant [5] exploits spatial carbon intensity differences across edge data centers. Ecomap [6] optimizes multi-tenant execution on edge servers with dynamic power thresholds, reporting 30% average carbon reduction. CarbonDIS [15] proposes carbon-aware scheduling for DNN inference on heterogeneous GPUs, incorporating grid carbon intensity into placement decisions, but models only computational energy, ignoring inter-node data transfer. System-level frameworks such as Carbon Containers [12] provide fine-grained carbon accounting for cloud workloads, but target datacenter rather than edge deployments. None of these approaches model transfer carbon.

DNN partitioning. Neurosurgeon [14] pioneered collaborative DNN inference between cloud and mobile devices, demonstrating latency and energy benefits through optimal split-point selection. HiDP [8] achieves 38% lower latency and 46% lower energy on Jetson and Raspberry Pi platforms through hierarchical partitioning. The energy savings come from faster execution, not carbon optimization. DynaSplit [9] co-optimizes split points and hardware for energy. It measures power rather than carbon intensity. CoEdge [10] reduces latency through cooperative inference but does not measure sustainability.

Gap. No existing framework jointly models compute carbon, transfer carbon, and multi-factor node selection. TRACE fills this gap.

III. SYSTEM MODEL

A. Node Profiles

Each node n_i in our cluster has five attributes: CPU quota α_i , RAM μ_i in GB, carbon intensity γ_i in gCO₂/kWh, peak

TABLE I
HETEROGENEOUS NODE PROFILES

Node	CPU	RAM	ρ_i (W)	γ_i (g/kWh)	Link
Node-6	0.6	1.0 GB	15	200	Fiber
Node-5	0.65	1.2 GB	15	250	Fiber
Node-4	0.7	1.4 GB	15	350	Fiber
Node-3	0.8	1.6 GB	18	450	WiFi
Node-2	0.9	1.8 GB	22	580	5G
Node-1	1.0	2.0 GB	25	620	4G/LTE

power ρ_i in watts, and network type τ_i . The network type is one of 4G/LTE, 5G, WiFi, or Fiber. Table I shows our six-node testbed.

B. Carbon Accounting

Each DNN model is split into three partitions, labeled P_0 (closest to input), P_1 (middle layers), and P_2 (closest to output). A partition P_k has compute cost f_k MFLOPs and output size o_k MB. Execution time t_k (ms) scales with MFLOP count and node CPU. Compute energy and carbon are:

$$E_{\text{comp}}(k, i) = \frac{\rho_i \times t_k}{3.6 \times 10^9} \text{ (kWh)}, \quad C_{\text{comp}}(k, i) = E_{\text{comp}} \times \gamma_i \quad (1)$$

where ρ_i is peak power (W), t_k is execution time (ms), and γ_i is grid carbon intensity (gCO₂/kWh). The divisor 3.6×10^9 converts watt-milliseconds to kWh.

Transfer energy depends on the sender's link type τ_s :

$$E_{\text{trans}}(k, s) = \frac{o_k}{1024} \times \epsilon(\tau_s) \text{ (kWh)} \quad (2)$$

where o_k is partition output (MB), 1024 converts MB to GB, and $\epsilon(\tau_s)$ is the energy coefficient of the sender's link type in kWh/GB. We use coefficients from Malmudin [7]: 0.14 for 4G, 0.08 for 5G, 0.08 for WiFi, and 0.04 kWh/GB for Fiber. Bandwidths $B(\tau)$ are 50 Mbps for 4G, 100 Mbps for 5G, 100 Mbps for WiFi, and 500 Mbps for Fiber. If the next partition stays on the same node, transfer energy is zero. Total carbon is:

$$C_{\text{total}}(n_i) = C_{\text{comp}}(i) + E_{\text{trans}}(s) \times \gamma_s \quad (3)$$

where γ_s is the sender node's grid carbon intensity.

IV. TRACE SCHEDULING

A. Multi-Factor Scoring

TRACE picks the best node in one pass. For each eligible node, it computes a weighted score:

$$S(n_i) = w_R s_R + w_L s_L + w_P s_P + w_B s_B + w_C s_C \quad (4)$$

The five sub-scores are: CPU and RAM availability (s_R), queue depth ($s_L = 1 - \text{load}$), estimated latency (s_P), bandwidth ratio (s_B), and normalized carbon (s_C). Each is computed per node using the node's attributes and current load. The carbon score uses Min-Max normalization:

Algorithm 1 TRACE Node Selection

Require: Partition P_k , nodes $\mathcal{N}$, sender n_s , weights $\mathbf{w}$

Ensure: Selected node n^*

```

1: for each  $n_i \in \mathcal{N}$  do
2:   if  $\mu_i \geq \mu_{\text{req}} \wedge \alpha_i \geq \alpha_{\text{min}}$  then
3:      $t_i \leftarrow c_{\text{base}} \times (f_k/100) \times (\alpha_{\text{max}}/\alpha_i)$ 
4:      $C_{\text{comp}}(i) \leftarrow (\rho_i \times t_i/3.6 \times 10^9) \times \gamma_i$ 
5:     if mode = TRACE then
6:        $C_{\text{trans}}(i) \leftarrow (o_k/1024) \times \epsilon(\tau_s) \times \gamma_s$ 
7:        $C(i) \leftarrow C_{\text{comp}}(i) + C_{\text{trans}}(i)$ 
8:        $T_{\text{est}} \leftarrow t_i + 8000 \times o_k/B(\tau_s)$ 
9:     else
10:       $C(i) \leftarrow C_{\text{comp}}(i); T_{\text{est}} \leftarrow t_i$ 
11:    end if
12:    Compute sub-scores  $s_R, s_L, s_P, s_B$ 
13:  end if
14: end for
15:  $C_{\text{min}} \leftarrow \min C(i); C_{\text{max}} \leftarrow \max C(i)$ 
16: for each candidate  $n_i$  do
17:    $s_C \leftarrow 1 - (C(i) - C_{\text{min}})/\max(C_{\text{max}} - C_{\text{min}}, \epsilon)$ 
18:    $S_i \leftarrow \mathbf{w} \cdot [s_R, s_L, s_P, s_B, s_C]^T$ 
19: end for
20: return  $n^* = \arg \max S_i$ 

```

$$s_C(n_i) = 1 - \frac{C_{\text{total}}(n_i) - C_{\text{min}}}{C_{\text{max}} - C_{\text{min}} + \epsilon} \quad (5)$$

where C_{min} and C_{max} are the smallest and largest carbon values among eligible nodes, and ϵ is a small constant preventing division by zero.

Table II lists the four mode configurations.

B. Scheduling Algorithm

Algorithm 1 shows the full procedure. For each partition, it filters nodes by resources, computes total carbon (with transfer cost in TRACE mode), normalizes carbon scores, and selects the highest-scoring node. In the algorithm, $c_{\text{base}} = 3.0$ is the base time (ms per 100 MFLOPs), α_{max} is the maximum CPU quota among all nodes, μ_{req} and α_{min} are the partition's RAM and CPU requirements, $B(\tau_s)$ is the bandwidth of the sender's link type (Mbps), T_{est} is estimated total latency including transfer, s_R, s_L, s_P, s_B are sub-scores for resources, queue, performance, and bandwidth, and the constant 8000 converts MB/Mbps to milliseconds (8 bits/byte $\times$ 1000 ms/s).

V. EVALUATION

A. Setup

We use a SimPy discrete-event simulation with six nodes (Table I). Each DNN model is split into three partitions (P_0 ,

TABLE II
SCHEDULING MODE WEIGHTS

Mode	w_R	w_L	w_P	w_B	w_C
Performance	0.25	0.25	0.30	0.15	0.05
Balanced	0.20	0.20	0.15	0.15	0.30
Green	0.15	0.15	0.10	0.10	0.50
TRACE	0.15	0.15	0.10	0.10	0.50

TABLE III
LATENCY AND PER-INFERENCE CARBON (HETERO 3-NODE, RATE=1)

Model	Mode	Latency (ms)	Carbon (gCO ₂ /inf)
MobileNetV2	Performance	61	0.0047
	Balanced	57	0.0034
	Green	54	0.0025
	TRACE	46	0.0019
ResNet-50	Performance	382	0.045
	Balanced	392	0.029
	Green	401	0.019
	TRACE	368	0.015
VGG16	Performance	1830	0.320
	Balanced	1730	0.230
	Green	1640	0.068
	TRACE	1560	0.044

P_1, P_2). The models cover different workloads: MobileNetV2 (~ 300 MFLOPs, network-light), ResNet-50 (~ 4100 MFLOPs, compute-heavy), and VGG16 (~ 15500 MFLOPs, transfer-heavy due to large tensors). All results are at 1 req/s (no queueing effects) on a 3-node subset (Node-1, Node-4, Node-6). Each run processes 500 inference requests. Node-1, Node-4, and Node-6 were selected to maximize diversity: Node-1 is the fastest CPU (1.0 core) but uses 4G/LTE with the highest carbon intensity; Node-4 is a mid-range Fiber-connected node; Node-6 is the lowest-carbon Fiber node.

B. Results

Table III shows average latency and per-inference carbon for all four modes.

Figure 1 visualizes this data as a grouped bar chart. The right panel uses a logarithmic scale on the carbon axis so that all modes, including TRACE, remain clearly visible.

MobileNetV2. Being network-light (~ 300 MFLOPs, small intermediate tensors), MobileNetV2’s latency is dominated by transfer overhead rather than compute. Performance mode spreads partitions across three nodes (Node-1, Node-4, Node-6), incurring two inter-node transfers, one of which is over 4G LTE. The 4G hop adds both latency (~ 20 ms transfer time) and transfer carbon. Concretely, routing P_2 ’s output (~ 33 KB) from Node-1 over 4G/LTE incurs approximately 0.0028 g of transfer carbon ($0.14 \text{ kWh/GB} \times 620 \text{ g/kWh}$). Balanced and Green modes avoid the 4G link but still pay Fiber-based transfer costs between nodes. TRACE consolidates all three partitions on Node-6 (Fiber, 15W, $\gamma_i = 200$), eliminating transfer entirely. Latency drops from 61 ms to 46 ms, a 24% improvement, because colocation removes network delay. The latency improvement follows from eliminating inter-node transfer through colocation, not from carbon scoring per se; a carbon-unaware scheduler that happened to consolidate on Node-6 would achieve similar speed. TRACE’s distinct contribution is jointly minimizing carbon while making this consolidation decision. Carbon falls from 0.0025 g (Green) to 0.0019 g per inference, a 24% additional reduction, as the transfer-aware scoring avoids residual cross-node transfers.

ResNet-50. This model is compute-heavy (~ 4100 MFLOPs). Compute time dominates total latency, so scheduling decisions have limited impact on speed. Performance mode routes P_0 to Node-1 (fastest CPU, 1.0 core, 4G/LTE) for quick computation but pays transfer carbon on the 4G link. TRACE avoids 4G entirely, placing partitions on Fiber-connected nodes (Node-4, Node-6). Latency is similar across modes (368–401 ms), as expected for a compute-bound workload. Green mode reduces carbon to 0.019 g per inference by favoring low-carbon-intensity nodes for compute. TRACE further reduces this to 0.015 g per inference, a 21% reduction over Green, by also accounting for transfer energy when placing partitions.

VGG16. VGG16’s large intermediate tensors (>100 MB per partition output) make it the most transfer-sensitive model. Performance mode distributes work across all three nodes, sending massive tensors over the 4G link, producing 0.320 g per inference. Green mode reduces this to 0.068 g by favoring low- γ_i Fiber nodes, but still pays transfer costs because it only models compute carbon. TRACE adds transfer awareness, consolidating partitions on Fiber-connected nodes and avoiding wireless links entirely. This brings carbon down to 0.044 g per inference, a 35% reduction beyond Green’s 0.068 g, by keeping large tensors off wireless links. Latency improves from 1830 ms (Performance) to 1560 ms (TRACE), a 15% gain, as the large tensors no longer traverse slow wireless links.

C. Discussion

Three findings stand out. First, the Transfer Trap is real. Schedulers that ignore network costs make routing decisions that hurt both latency and carbon. Second, transfer awareness tends to consolidate work on Fiber nodes, which also avoids bandwidth bottlenecks on wireless links. However, this effect and its impact on load distribution require further study at higher request rates. Third, the benefit varies by model. Network-light models (MobileNetV2) gain most in latency. Compute-heavy models (ResNet-50) show similar latency with lower carbon. Transfer-heavy models (VGG16) improve on both. These results are based on a single configuration (3

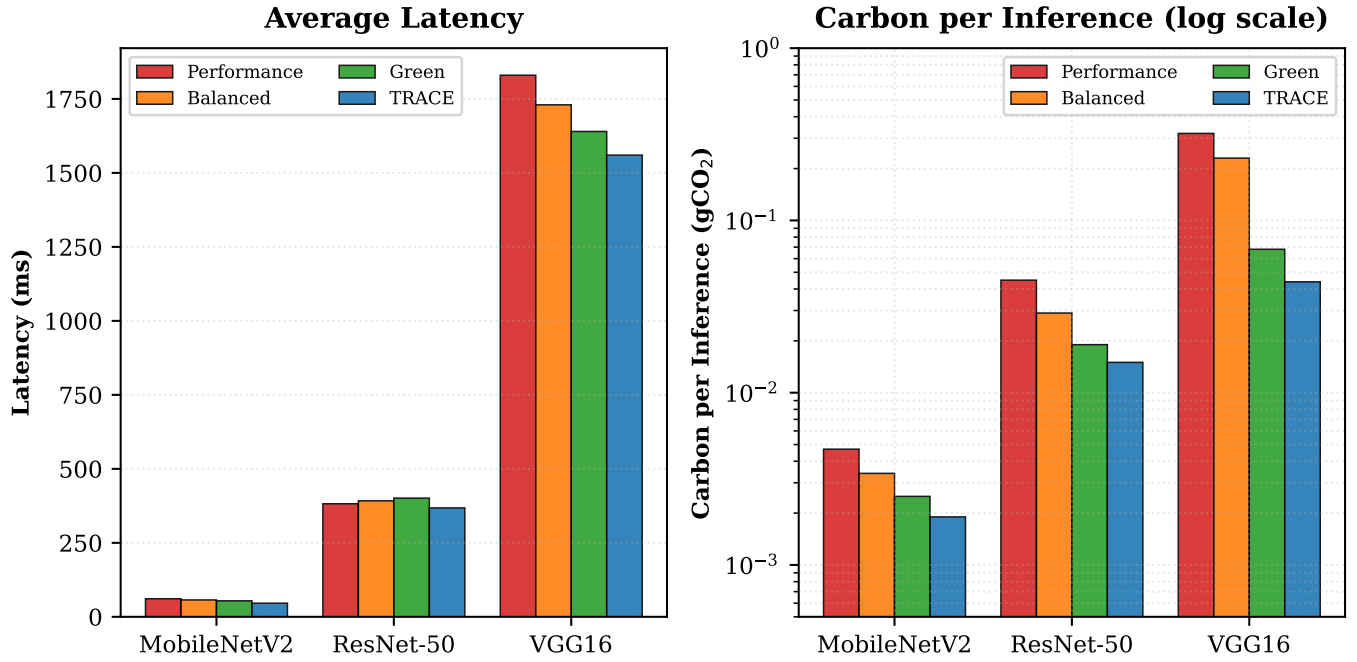


Fig. 1. Bar chart comparison of latency (left) and per-inference carbon (right, log scale) across four scheduling modes for three DNN models at 1 req/s.

partitions, 1 req/s); generalization to higher request rates and different partition granularities requires further investigation.

VI. CONCLUSION

We presented TRACE, a scheduler that models both compute and transfer carbon for partitioned DNN inference. It uses per-link transfer coefficients, Min-Max normalization, and a local execution rule to avoid the Transfer Trap. In a SimPy simulation on a six-node heterogeneous testbed with real PyTorch profiling, TRACE reduces latency for network-light models and keeps per-inference carbon consistently low across three model architectures. These results show that the trade-off between performance and sustainability can be a co-optimization when transfer costs are properly accounted for.

REFERENCES

- [1] Y. Mao *et al.*, “A survey on mobile edge computing: The communication perspective,” *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, 2017.
- [2] D. Patterson *et al.*, “Carbon emissions and large neural network training,” arXiv:2104.10350, 2021.
- [3] H. Ke, W. Jin, and H. Wang, “CarbonCP: Carbon-aware DNN partitioning with conformal prediction,” arXiv:2404.16970, 2024.
- [4] G. Zhang *et al.*, “CarbonEdge: Carbon-aware deep learning inference framework for sustainable edge computing,” arXiv:2603.27420, 2026.
- [5] L. Wu, W. A. Hanafy, A. Souza, K. Nguyen, J. Harkes, D. Irwin, and M. Satyanarayanan, “CarbonEdge: Leveraging mesoscale spatial carbon-intensity variations for low carbon edge computing,” in *Proc. ACM HPDC*, 2025.
- [6] V. Paramanayakam, A. Karatzas, D. Stamoulis, and I. Anagnostopoulos, “Ecomap: Sustainability-driven optimization of multi-tenant DNN execution on edge servers,” arXiv:2503.04148, 2025.
- [7] J. Malmmodin and D. Lundén, “The energy and carbon footprint of the global ICT and E&M sectors,” *Sustainability*, vol. 10, no. 9, p. 3027, 2018.
- [8] Z. Taufique *et al.*, “HiDP: Hierarchical DNN partitioning for distributed inference,” in *Proc. IEEE DATE*, 2025.
- [9] D. May, A. Tundo, S. Ilager, and I. Brandic, “DynaSplit: A hardware-software co-design framework for energy-aware inference on edge,” arXiv:2410.23881, 2024.
- [10] L. Zeng *et al.*, “CoEdge: Cooperative DNN inference with adaptive workload partitioning,” *IEEE/ACM Trans. Netw.*, vol. 29, no. 2, 2021.
- [11] E. Strubell, A. Ganesh, and A. McCallum, “Energy and policy considerations for deep learning in NLP,” in *Proc. ACL*, pp. 3645–3650, 2019.
- [12] J. Thiede, N. Bashir, D. Irwin, and P. Shenoy, “Carbon Containers: A system-level facility for managing application-level carbon emissions,” in *Proc. ACM Symp. Cloud Comput. (SoCC)*, pp. 17–31, 2023.
- [13] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, “Green AI,” *Commun. ACM*, vol. 63, no. 12, pp. 54–63, 2020.
- [14] Y. Kang *et al.*, “Neurosurgeon: Collaborative intelligence between the cloud and mobile edge,” in *Proc. ACM ASPLOS*, pp. 615–629, 2017.
- [15] S. M. Nabavinejad, W. Jia, S. Taneja, and T. Guo, “CarbonDIS: Carbon-aware DNN inference scheduling on heterogeneous GPUs,” in *Proc. IEEE CCGrid*, pp. 315–324, 2025.